Laboratorio JavaScript

Tecnologie Web

Dal blog multipagina a una SPA in JavaScript puro

Giovanni Delnevo - giovanni.delnevo@unibo.it

Obiettivi del laboratorio

- Caricare dati JSON esterni con `fetch()` e `async/await`
- Utilizzare le classi `Articolo`, `Autore` e `Autori` già realizzate
- Costruire un router basato sull'hash dell'URL
- Aggiornare il contenuto della pagina senza ricaricare il documento HTML
- Gestire viste, errori e rotte inesistenti

Materiale iniziale

```
blog/
├── index.html
├── css/style.css
├── img/
├── data/
│   ├── articoli.json
│   └── autori.json
└── js/
    ├── model.js     # completo
    ├── data.js      # da completare
    ├── views.js     # parziale
    └── router.js    # da completare
```

Avvio:

- Spostare la cartella `./xampp/htdocs` (nei PC di laboratorio `C:\xampp\htdocs`).
- Visitare `localhost/04-lab-materiale`.



Perché più file JavaScript?

`model.js`

Classi e metodi `toDOM()` già sviluppati.

`data.js`

Richieste HTTP e conversione JSON in oggetti.

Ogni file ha una responsabilità distinta.

`views.js`

Costruzione e aggiornamento del DOM.

`router.js`

Scelta della vista in base all'URL.

Moduli JavaScript: import ed export

```
// model.js
export class Articolo {
    // ...
}
```

```
// data.js
import { Articolo } from "./model.js";
```

Nel documento HTML:

```
<script type="module" src="js/router.js"></script>
```

Gli script `type="module"` vengono eseguiti dopo il parsing dell'HTML: **non serve** `defer`.



Esercizio 1: il router

Obiettivo: cambiare vista senza caricare un nuovo documento HTML.

- Un solo `index.html`
- Un contenitore centrale: `<main id="app"></main>`
- Link del menu basati su hash
- Una funzione `router()` che interpreta l'URL
- Evento `hashchange` per intercettare la navigazione

```
<a href="#/">Home</a>
<a href="#/archivio">Archivio</a>
<a href="#/contatti">Contatti</a>
<a href="#/login">Login</a>
```

L’hash identifica la vista

URL	Vista
`#/`	Home
`#/archivio`	Archivio
`#/contatti`	Contatti
`#/login`	Login
`#/articolo/3`	Dettaglio dell’articolo 3
`#/qualcosa`	Pagina non trovata

```
const hash = window.location.hash || "#/";
```

La parte dopo `#` può cambiare senza richiedere un nuovo documento HTML.



Esercizio 1

Router:

- In `js/router.js`, sostituire il comportamento provvisorio.
- Quando dobbiamo chiamare la funzione `router`?

Caricamento Dati:

- Nel precedente laboratorio, il JSON era incorporato nel codice JavaScript.
- In questo laboratorio, il JSON viene recuperato con una richiesta HTTP.
- Implementiamo `caricaArticoli()` in `data.js`

Gestione della Vista:

- In `views.js`, completare `mostraHome()`.

Esercizio 2: Archivio

Obiettivo: completare la vista `#/archivio`.

In `views.js`, implementare `mostraArchivio()`:

1. Mostrare il titolo e lo stato di caricamento
2. Richiamare `caricaArticoli()`
3. Visualizzare **tutti** gli articoli usando `toDOM()`
4. Gestire array vuoto ed errori con `try/catch`

Suggerimento: partire da `mostraHome()` e individuare cosa cambia.

Esercizio 3: Contatti

Obiettivo: completare la vista `#/contatti`.

In `data.js`:

- Implementare `caricaAutori()`
- Recuperare `./data/autori.json`
- Convertire ogni elemento in un'istanza di `Autore`
- Restituire un'istanza della classe `Autori`

In `views.js`:

- Completare `mostraContatti()`
- Aggiungere al DOM la tabella prodotta da `autori.toDOM()`
- Gestire caricamento, array vuoto ed errori

Esercizio 4: dettaglio articolo

Obiettivo: gestire rotte come `#/articolo/3`.

In `router.js`:

```
if (hash.startsWith("#/articolo/")) {
    const parti = hash.split("/");
    const id = Number(parti[2]);

    if (parti.length === 3 &&
        Number.isSafeInteger(id) && id > 0) {
        mostraArticolo(parti[2]);
    } else {
        mostraNonTrovata();
    }
}
```

Attenzione: integrare questo ramo prima dell'`else` finale del router.

Esercizio 4: mostraArticolo(id)

In `views.js`:

1. Caricare gli articoli con `caricaArticoli()`
2. Cercare l'articolo richiesto con `find()`
3. Visualizzare il contenuto completo con `toDOM(true)`
4. Gestire un ID non presente nei dati
5. Aggiungere il link `#/archivio` per tornare all'elenco
6. Gestire gli errori di caricamento

Suggerimento: confrontare `String(a.id)` con il parametro `id`.

Login: una vista già disponibile

La funzione `mostraLogin()` è già implementata in `views.js`.

- Crea il form con username e password
- Riutilizza la validazione del primo laboratorio
- Intercetta `submit` con `preventDefault()`
- Mostra gli errori accanto ai campi
- Non effettua alcuna autenticazione server-side

Verifica: la navigazione verso `#/login` deve mostrare il form senza caricare un nuovo documento HTML.

Test finale della SPA

- ☐ `#/` mostra gli ultimi due articoli
- ☐ `#/archivio` mostra tutti gli articoli
- ☐ `#/contatti` mostra la tabella degli autori
- ☐ `#/login` mostra il form con validazione
- ☐ `#/articolo/1` mostra l'articolo completo
- ☐ Un articolo inesistente viene gestito
- ☐ Una rotta sconosciuta mostra la pagina 404
- ☐ Indietro e Avanti funzionano
- ☐ La navigazione non ricarica `index.html`

Cosa abbiamo imparato?

- Il router deve interpretare l'URL e scegliere la vista
- Ogni vista deve costruire e aggiornare manualmente il DOM
- I dati arrivano in modo asincrono
- Navigazione, caricamento ed eventi devono rimanere coordinati

Nelle prossime lezioni: Vue.js e Vue Router permetteranno di gestire queste responsabilità con componenti, reattività e routing dichiarativo.